Maze Generation with Prim's Algorithm and Maze Solving with A* Search Algorithm

Dinanath Padhya*, Krishna Acharya†
*THA078BEI010 †THA078BEI020
Department of Electronics, Communication and Information Engineering
Thapathali Campus, IOE, Tribhuvan University, Nepal

***Abstract*—Maze generation and pathfinding are essential problems in the field of Artificial Intelligence, particularly in robotics, gaming, and autonomous navigation systems. In this lab, we implement a complete pipeline to generate and solve mazes using two classical algorithms. The maze is generated using a randomized version of Prim's algorithm, which ensures the creation of a perfect maze, one with a unique solution and no cycles. Once generated, the maze is solved using the A* (A-Star) search algorithm, a widely used pathfinding technique that combines the benefits of uniform-cost search and heuristics to efficiently find optimal paths. We have developed Python-based implementations for both algorithms and used two visualization approaches, matplotlib for static/animated plotting and Pygame for interactive stepwise visualization. Finally, we analyze performance using metrics such as total nodes visited, total path length, and time taken for the solution, providing a comprehensive understanding of the search dynamics in generated mazes.**

I. Introduction

Pathfinding and maze generation form the core of many artificial intelligence applications, including robotics, autonomous vehicles, and game development. Efficient generation and solving of mazes are crucial not only for entertainment but also for testing AI navigation capabilities under constrained environments.

This lab introduces two widely known algorithms in AI: Prim's algorithm for maze generation and the A* search algorithm for pathfinding. Prim's algorithm is a randomized, graph-based algorithm originally intended for finding minimum spanning trees, but with slight modifications, it can generate perfect mazes, mazes with no loops and a single solution path between any two points. This approach ensures that the maze has a structured yet unpredictable topology suitable for testing search algorithms.

Once the maze is constructed, we solve it using the A* algorithm, which is known for its optimal and complete nature when a consistent heuristic is used. A* operates using a best-first approach that balances the cost to reach a node (g-cost) and an estimated cost to the goal (h-cost), forming a total cost (f-cost = g + h). We use Manhattan distance as the heuristic due to the grid-based nature of the maze.

To enhance understanding and provide real-time feedback, we incorporated two visualization strategies: static/animated visualization using matplotlib and interactive visualization using Pygame. These help in observing the internal behavior of the search process and better interpret performance characteristics.

Through this lab, we aim to not only implement and compare these algorithms but also provide insight into their behavior, efficiency, and practical applications. The experiment reinforces core AI concepts such as heuristic search, graph traversal, and complexity trade-offs in algorithm design.

II. Methodology

This lab follows a structured pipeline that begins with maze generation using Prim's algorithm, followed by the placement of start and goal positions. The generated maze is then solved using the A* search algorithm, and the entire process is visualized through both static/animated plots and an interactive GUI. Prim's algorithm ensures the maze is a perfect maze, i.e., there is exactly one path between any two points. The A* algorithm efficiently finds the optimal path between the start and goal using a heuristic-based approach. For implementation, we used Python as the programming language along with libraries such as NumPy for numerical operations, matplotlib for plotting and animation, and Pygame for building an interactive visual interface.

A. Maze Generation Using Prim's Algorithm

1) Initialization: We begin by initializing the maze as a 2D grid where all cells are set to walls (value 1). A random odd-indexed cell is chosen as the starting point and converted into a passage (value 0). This ensures that walls and passages are properly aligned in a grid with odd dimensions.

2) Wall Selection and Carving: From the starting cell, all adjacent walls (in four cardinal directions) are added to a frontier list. The algorithm proceeds by randomly selecting a wall from the frontier. If the cell on the opposite side of that wall is still a wall and has exactly one neighboring passage, the wall and the opposite cell are converted into passages. This process repeats until there are no more valid frontier walls.

3) Properties: This randomized wall-growing approach ensures that:

- The resulting maze is **perfect** shown in Figure 1, there exists a unique path between any two points with no loops.
- Each run of the algorithm produces a different maze structure due to random wall selection.

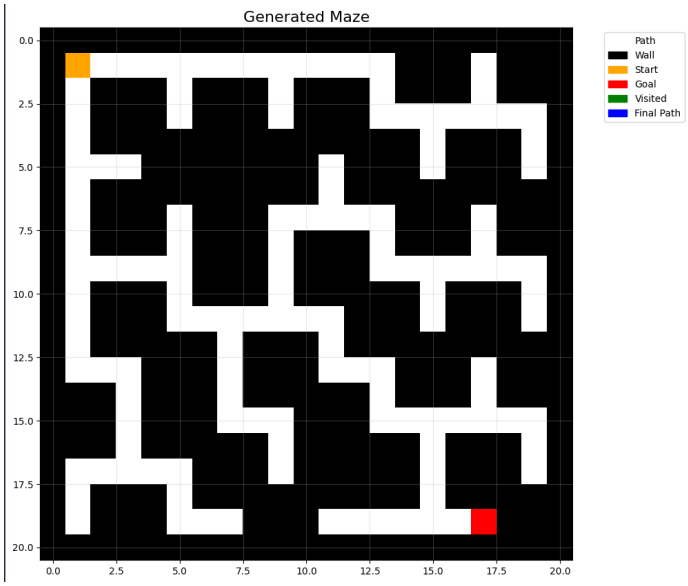


Fig. 1. Initial Maze Generation using Prim's Algorithm

Algorithm 1 Maze Generation using Prim's Algorithm

procedure GENERATEMAZE(rows, cols)
  if rows or cols even **then**
    Increment to make them odd
  end if
  Initialize maze grid with all walls (1)
  Select random odd cell (r, c) as start; set $maze[r][c] \leftarrow 0$
  $walls \leftarrow$ adjacent walls of (r, c)
  while $walls$ is not empty **do**
    Pick random wall (r_1, c_1, r_2, c_2) from $walls$
    if $maze[r_2][c_2] = 1$ and only one adjacent cell to (r_2, c_2) is a passage **then**
      $maze[(r_1 + r_2)/2][(c_1 + c_2)/2] \leftarrow 0$ ▷ Carve wall
      $maze[r_2][c_2] \leftarrow 0$
      Add new walls from (r_2, c_2) to $walls$
    end if
    Remove (r_1, c_1, r_2, c_2) from $walls$
  end while
  return maze
end procedure

B. Start and Goal Placement

After generating the maze using Prim's algorithm, we assign designated positions for the starting and goal points. According to the lab guidelines, the start point is placed at cell $(1, 1)$ and the goal point at cell $(rows - 2, cols - 2)$, both of which correspond to the first and last accessible inner positions of the maze grid.

To distinguish these points visually and programmatically, we encode them with unique integer values:

- Start position $(1, 1)$ is assigned the value **2**
- Goal position $(rows - 2, cols - 2)$ is assigned the value **3**

In the event that the default positions are blocked by walls, the algorithm searches for the next nearest valid passage cell in the respective direction (top-left for start, bottom-right for goal) and assigns the labels accordingly.

Function: `place_start_goal(maze)` The output maze looks like as in Figure 1.

Returns: A modified NumPy array with encoded start and goal positions.

C. A* Search Algorithm Implementation

1) Cost Function: The A* search algorithm uses a cost function to determine the most promising node to explore next:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the actual cost from the start node to the current node n.
- $h(n)$ is the heuristic cost from node n to the goal. In our implementation, this is computed using the Manhattan distance:

$$h(n) = |x_{goal} - x_n| + |y_{goal} - y_n|$$

2) Algorithm Steps: The A* algorithm can be formally described as follows:

Algorithm 2 A* Search Algorithm

procedure ASTARSEARCH(maze, start, goal)
  Initialize open set as a priority queue containing start node with $f(start)$
  Initialize closed set as empty
  while open set is not empty **do**
    $current \leftarrow$ node in open set with lowest f value
    if $current = goal$ **then**
      return reconstructed path from start to goal
    end if
    Remove $current$ from open set
    Add $current$ to closed set
    for all neighboring nodes n of $current$ **do**
      if n is a wall or n in closed set **then**
        continue
      end if
      $g_{tentative} \leftarrow g(current) + 1$
      if n not in open set or $g_{tentative} < g(n)$ **then**
        Set parent of n to $current$
        $g(n) \leftarrow g_{tentative}$
        $h(n) \leftarrow$ heuristic estimate to goal (e.g., Manhattan distance)
        $f(n) \leftarrow g(n) + h(n)$
        if n not in open set **then**
          Add n to open set
        end if
      end if
    end for
  end while
  return failure (no path found)
end procedure

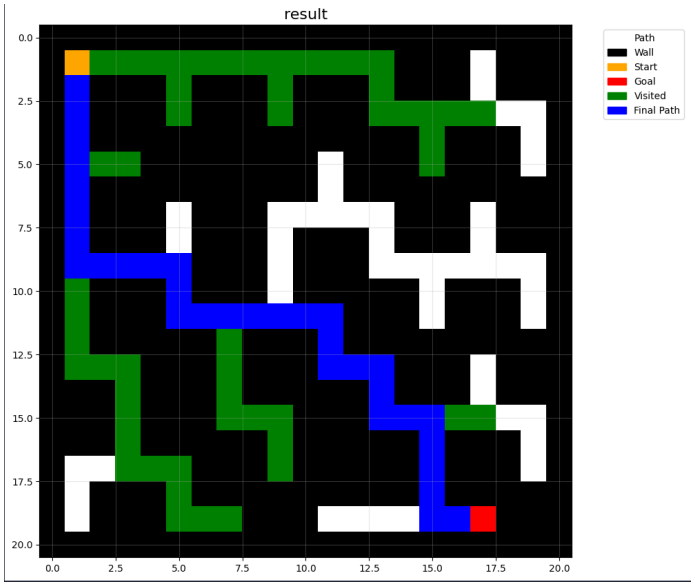


Fig. 2. Maze Solved using A* Search Algorithm

TABLE I
PERFORMANCE METRICS

Maze Size	Total Nodes Visited	Path Length	Execution Time (s)
20×20	87	35	0.002
10×10	45	15	0.001

The algorithm maintains two sets: an open set for nodes to be evaluated and a closed set for nodes already evaluated. It iteratively explores the most promising node based on the lowest f value, expanding its neighbors and updating their costs until it finds the goal or exhausts all possibilities. Figure 2 illustrates the maze after applying the A* algorithm, showing the path from start to goal.

3) Metrics Recorded: To analyze performance and efficiency, we record the following:

- **Total nodes visited:** The count of all expanded/visited cells.
- **Path length:** The number of cells in the optimal path.
- **Execution time:** Total time taken to compute the solution path.

D. Visualization Techniques

1) Matplotlib Static/Animated Visualization: We used matplotlib to generate both static and animated visualizations of the maze-solving process. The maze is rendered with color coding for:

- White – Passable path
- Black – Walls
- Orange – Start node
- Red – Goal node
- Green – Visited nodes
- Blue – Final solution path

2) Pygame Interactive Interface: For a more engaging and controlled experience, we developed an interactive GUI using Pygame as shown in Figure 3. The interface provides:

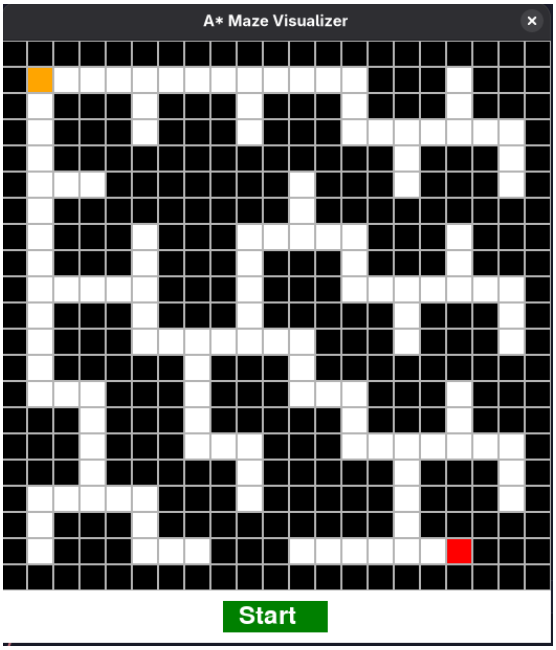


Fig. 3. Pygame Interactive Visualization of Maze Solving

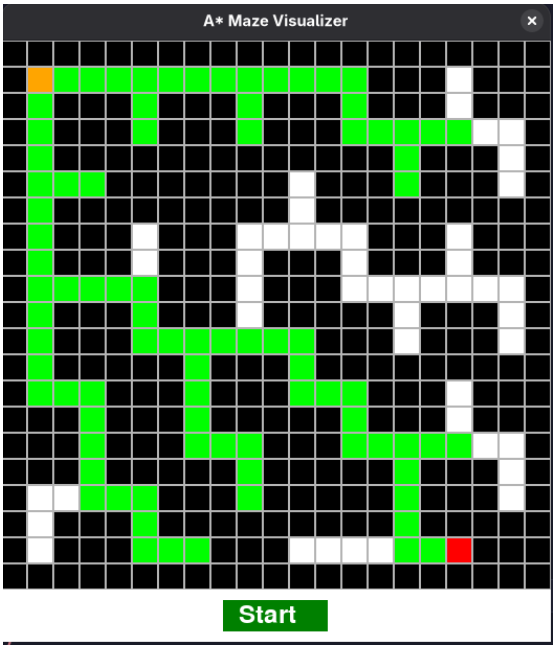


Fig. 4. Pygame Interactive Visualization of Maze Solving (Step-by-Step)

- A 'Start' button to begin the solving animation.
- Real-time display of the current search node.
- Step-by-step progression through each search iteration.
- A final view highlighting the completed path from start to goal as shown in Figure 4.

III. RESULTS AND DISCUSSION

The maze generation and solving process was successfully implemented, yielding various mazes and their respective solutions. The A* algorithm efficiently found paths in all generated mazes, demonstrating its effectiveness in grid-based

pathfinding tasks. The performance metrics collected during the experiments are summarized in Table I. The total nodes visited varied with maze complexity, while the path length remained consistent with the optimal solution characteristics of A*. The execution time was generally low, indicating the efficiency of the A* algorithm in this context.

IV. Conclusion

This lab successfully demonstrated the generation of complex mazes and its efficient solution using A*. The heuristic-driven approach ensured optimal paths. The addition of Pygame visualization made the search process intuitive and engaging. Future enhancements may include support for DFS/BFS comparisons and weighted grid paths.